

PRACTICAL WEEK-7

Task-1: Write a program that prints the addresses of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.

```
madhulika@LAPTOP-D2M8NKBH:~$ cd ~
madhulika@LAPTOP-D2M8NKBH:~$ mkdir memory_task
madhulika@LAPTOP-D2M8NKBH:~$ cd memory_task
madhulika@LAPTOP-D2M8NKBH:~/memory_task$ pwd
/home/madhulika/memory_task
madhulika@LAPTOP-D2M8NKBH:~/memory_task$ nano memory_layout.c

madhulika@LAPTOP-D2M8NKBH:~/memory_task$ gcc memory_layout.c -o memory_layout
madhulika@LAPTOP-D2M8NKBH:~/memory_task$ ls
memory_layout  memory_layout.c
madhulika@LAPTOP-D2M8NKBH:~/memory_task$ ./memory_layout
Linux Process Memory Address Layout
-----
Code   address : 0x5eb5e53e81c9
Global address : 0x5eb5e53eb010
Static address : 0x5eb5e53eb014
Heap   address : 0x5eb5f8312010
Stack  address : 0x7ffe5d46335c

Values:
Global = 100
Static = 200
Heap   = 400
Stack  = 300
madhulika@LAPTOP-D2M8NKBH:~/memory_task$ ps
  PID TTY          TIME CMD
  340 pts/0        00:00:00 bash
  626 pts/0        00:00:00 ps
madhulika@LAPTOP-D2M8NKBH:~/memory_task$ ./memory_layout &
[1] 642
Linux Process Memory Address Layout
-----
Code   address : 0x5d5249c9b1c9
Global address : 0x5d5249c9e010
Static address : 0x5d5249c9e014
Heap   address : 0x5d5262a8f010
Stack  address : 0x7fff6e8191dc

Values:
Global = 100
Static = 200
```

```

Values:
Global = 100
Static = 200
Heap = 400
Stack = 300
[1]+ Done ./memory_layout
madhulika@LAPTOP-D2M8NKBH:~/memory_task$ ps
  PID TTY          TIME CMD
  340 pts/0    00:00:00 bash
  654 pts/0    00:00:00 ps
madhulika@LAPTOP-D2M8NKBH:~/memory_task$ nano memory_layout.c
madhulika@LAPTOP-D2M8NKBH:~/memory_task$ nano memory_layout.c
madhulika@LAPTOP-D2M8NKBH:~/memory_task$ gcc memory_layout.c -o memory_layout
memory_layout.c: In function 'main':
memory_layout.c:43:1: error: implicit declaration of function 'sleep' [-Wimplicit-function-declaration]
  43 | sleep(60);
    | ^~~~~~
memory_layout.c:47:1: error: expected declaration or statement at end of input
  47 | return 0;
    | ^~~~~~
madhulika@LAPTOP-D2M8NKBH:~/memory_task$ nano memory_layout.c
madhulika@LAPTOP-D2M8NKBH:~/memory_task$ gcc memory_layout.c -o memory_layout
madhulika@LAPTOP-D2M8NKBH:~/memory_task$ ./memory_layout
Linux Process Memory Address Layout
-----
Code   address : 0x5884d0fad1e9
Global address : 0x5884d0fb0010
Static address : 0x5884d0fb0014
Heap   address : 0x5884e72e3010
Stack  address : 0x7fff9178cfbc

Values:
Global = 100
Static = 200
Heap = 400
Stack = 300

Process is running for memory analysis...
madhulika@LAPTOP-D2M8NKBH:~/memory_task$

```

```

#include <stdio.h>
#include <stdlib.h>

int global_var = 100;

void code_function()
{
    printf("This is a code segment function.\n");
}

int main()
{
    static int static_var = 200;

    int stack_var = 300;

    int *heap_var = (int *)malloc(sizeof(int));

    if (heap_var == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }

    *heap_var = 400;

    printf("Linux Process Memory Address Layout\n");
    printf("-----\n");

    printf("Code   address : %p\n", (void *)code_function);
    printf("Global address : %p\n", (void *)&global_var);
    printf("Static address : %p\n", (void *)&static_var);
    printf("Heap   address : %p\n", (void *)heap_var);
    printf("Stack  address : %p\n", (void *)&stack_var);

    printf("\nValues:\n");
    printf("Global = %d\n", global_var);

```

```

int stack_var = 300;

int *heap_var = (int *)malloc(sizeof(int));

if (heap_var == NULL)
{
    printf("Memory allocation failed.\n");
    return 1;
}

*heap_var = 400;

printf("Linux Process Memory Address Layout\n");
printf("-----\n");

printf("Code    address : %p\n", (void *)code_function);
printf("Global  address : %p\n", (void *)&global_var);
printf("Static  address : %p\n", (void *)&static_var);
printf("Heap    address : %p\n", (void *)heap_var);
printf("Stack   address : %p\n", (void *)&stack_var);

printf("\nValues:\n");
printf("Global = %d\n", global_var);
printf("Static = %d\n", static_var);
printf("Heap   = %d\n", *heap_var);
printf("Stack  = %d\n", stack_var);

printf("\nProcess is running for memory analysis...\n");
sleep(60);

free(heap_var);

return 0;

```

```

madhulika@LAPTOP-D2M8NKBH:~$ cd ~/memory_task
madhulika@LAPTOP-D2M8NKBH:~/memory_task$ ps -ef | grep memory_layout
madhuli+   811    340  0 03:34 pts/0    00:00:00 ./memory_layout
madhuli+   868    821  0 03:35 pts/2    00:00:00 grep --color=auto memory_layout
madhulika@LAPTOP-D2M8NKBH:~/memory_task$ cat /proc/811/maps
cat: /proc/811/maps: No such file or directory
madhulika@LAPTOP-D2M8NKBH:~/memory_task$ ./memory_layout &
[1] 900
Linux Process Memory Address Layout
-----
Code    address : 0x5a3feb0b31e9
Global  address : 0x5a3feb0b6010
Static  address : 0x5a3feb0b6014
Heap    address : 0x5a402554f010
Stack   address : 0x7ffd4f92731c

Values:
Global = 100
Static = 200
Heap   = 400
Stack  = 300

Process is running for memory analysis...
madhulika@LAPTOP-D2M8NKBH:~/memory_task$ ps -ef | grep memory_layout

```

```

madhulika@LAPTOP-D2M8NKBH: ~/memory_task$ pgrep memory_layout
900
madhulika@LAPTOP-D2M8NKBH: ~/memory_task$ cat /proc/900/maps
5a3feb0b2000-5a3feb0b3000 r--p 00000000 08:30 41350 /home/madhulika/memory_task/memory_layout
5a3feb0b3000-5a3feb0b4000 r-xp 00001000 08:30 41350 /home/madhulika/memory_task/memory_layout
5a3feb0b4000-5a3feb0b5000 r--p 00002000 08:30 41350 /home/madhulika/memory_task/memory_layout
5a3feb0b5000-5a3feb0b6000 r--p 00002000 08:30 41350 /home/madhulika/memory_task/memory_layout
5a3feb0b6000-5a3feb0b7000 rw-p 00003000 08:30 41350 /home/madhulika/memory_task/memory_layout
5a402554f000-5a4025570000 rw-p 00000000 00:00 0 [heap]
7866f3800000-7866f3828000 r--p 00000000 08:30 13628 /usr/lib/x86_64-linux-gnu/libc.so.6
7866f3828000-7866f39c0000 r-xp 00028000 08:30 13628 /usr/lib/x86_64-linux-gnu/libc.so.6
7866f39c0000-7866f3a0e000 r--p 001c0000 08:30 13628 /usr/lib/x86_64-linux-gnu/libc.so.6
7866f3a0e000-7866f3a12000 r--p 0020d000 08:30 13628 /usr/lib/x86_64-linux-gnu/libc.so.6
7866f3a12000-7866f3a14000 rw-p 00211000 08:30 13628 /usr/lib/x86_64-linux-gnu/libc.so.6
7866f3a14000-7866f3a21000 rw-p 00000000 00:00 0
7866f3a79000-7866f3a7c000 rw-p 00000000 00:00 0
7866f3a82000-7866f3a84000 rw-p 00000000 00:00 0
7866f3a84000-7866f3a88000 r--p 00000000 00:00 0 [vvar]
7866f3a88000-7866f3a8a000 r--p 00000000 00:00 0 [vvar_vclock]
7866f3a8a000-7866f3a8c000 r-xp 00000000 00:00 0 [vdso]
7866f3a8c000-7866f3a8d000 r--p 00000000 08:30 13625 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7866f3a8d000-7866f3abc000 r-xp 00001000 08:30 13625 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7866f3abc000-7866f3ac7000 r--p 00030000 08:30 13625 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7866f3ac7000-7866f3ac9000 r--p 0003b000 08:30 13625 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7866f3ac9000-7866f3aca000 rw-p 0003d000 08:30 13625 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7866f3aca000-7866f3acb000 rw-p 00000000 00:00 0
7ffd4f909000-7ffd4f92a000 rw-p 00000000 00:00 0 [stack]
madhulika@LAPTOP-D2M8NKBH: ~/memory_task$ grep -E 'memory_layout|heap|stack' /proc/900/maps
5a3feb0b2000-5a3feb0b3000 r--p 00000000 08:30 41350 /home/madhulika/memory_task/memory_layout
5a3feb0b3000-5a3feb0b4000 r-xp 00001000 08:30 41350 /home/madhulika/memory_task/memory_layout
5a3feb0b4000-5a3feb0b5000 r--p 00002000 08:30 41350 /home/madhulika/memory_task/memory_layout
5a3feb0b5000-5a3feb0b6000 r--p 00002000 08:30 41350 /home/madhulika/memory_task/memory_layout
5a3feb0b6000-5a3feb0b7000 rw-p 00003000 08:30 41350 /home/madhulika/memory_task/memory_layout
5a402554f000-5a4025570000 rw-p 00000000 00:00 0 [heap]
7ffd4f909000-7ffd4f92a000 rw-p 00000000 00:00 0 [stack]
madhulika@LAPTOP-D2M8NKBH: ~/memory_task$

```

Observation: The program successfully displayed the addresses of code, global, static, heap, and stack variables. The `/proc/900/maps` file was used to analyze the Linux process address space. The code segment contains executable instructions, global and static variables are stored in the data segment, dynamically allocated memory is stored in the heap, and local variables are stored in the stack. The virtual addresses may change between executions due to ASLR.

Task-2: Make use of the /proc/<PID>/maps file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

```
madhulika@LAPTOP-D2M8NKBH:~/memory_task$ pgrep memory_layout
[1]+  Done                  ./memory_layout
madhulika@LAPTOP-D2M8NKBH:~/memory_task$ ./memory_layout &
PID=$!
echo "PID = $PID"
[1] 1249
Linux Process Memory Address Layout
-----
Code   address : 0x60de45d4b1e9
Global address : 0x60de45d4e010
Static address : 0x60de45d4e014
Heap   address : 0x60de51882010
Stack  address : 0x7ffd7d8c8bcc

Values:
Global = 100
Static = 200
Heap   = 400
Stack  = 300

Process is running for memory analysis...
PID = 1249
madhulika@LAPTOP-D2M8NKBH:~/memory_task$ cat /proc/1020/maps
cat: /proc/1020/maps: No such file or directory
madhulika@LAPTOP-D2M8NKBH:~/memory_task$ cat /proc/1249/maps
60de45d4a000-60de45d4b000 r--p 00000000 08:30 41350 /home/madhulika/memory_task/memory_layout
60de45d4b000-60de45d4c000 r-xp 00001000 08:30 41350 /home/madhulika/memory_task/memory_layout
60de45d4c000-60de45d4d000 r--p 00002000 08:30 41350 /home/madhulika/memory_task/memory_layout
60de45d4d000-60de45d4e000 r--p 00002000 08:30 41350 /home/madhulika/memory_task/memory_layout
60de45d4e000-60de45d4f000 rw-p 00003000 08:30 41350 /home/madhulika/memory_task/memory_layout
60de51882000-60de518a3000 rw-p 00000000 00:00 0 [heap]
779d8fe00000-779d8fe28000 r--p 00000000 08:30 13628 /usr/lib/x86_64-linux-gnu/libc.so.6
779d8fe28000-779d8ffc0000 r-xp 00028000 08:30 13628 /usr/lib/x86_64-linux-gnu/libc.so.6
779d8ffc0000-779d9000e000 r--p 001c0000 08:30 13628 /usr/lib/x86_64-linux-gnu/libc.so.6
779d9000e000-779d90012000 r--p 0020d000 08:30 13628 /usr/lib/x86_64-linux-gnu/libc.so.6
779d90012000-779d90014000 rw-p 00211000 08:30 13628 /usr/lib/x86_64-linux-gnu/libc.so.6
779d90014000-779d90021000 rw-p 00000000 00:00 0
779d901a9000-779d901ac000 rw-p 00000000 00:00 0
779d901b2000-779d901b4000 rw-p 00000000 00:00 0
779d901b4000-779d901b8000 r--p 00000000 00:00 0
779d901b8000-779d901ba000 r--p 00000000 00:00 0 [vvar]
779d901ba000-779d901ba000 r--p 00000000 00:00 0 [vvar_vclock]
```

```
madhulika@LAPTOP-D2M8NKBH:~/memory_task$ ./memory_layout & PID=$!; echo "PID = $PID"; sleep 1; pmap $PID; echo "----- IMPORTANT MAPPINGS -----"; grep -E 'me
mory_layout|heap|stack' /proc/$PID/maps
[1] 1333
PID = 1333
Linux Process Memory Address Layout
-----
Code   address : 0x566634ba51e9
Global address : 0x566634ba8010
Static address : 0x566634ba8014
Heap   address : 0x56666627e010
Stack  address : 0x7ffe397bf72c

Values:
Global = 100
Static = 200
Heap   = 400
Stack  = 300

Process is running for memory analysis...
1333: ./memory_layout
0000566634ba4000 4K r---- memory_layout
0000566634ba5000 4K r-x-- memory_layout
0000566634ba6000 4K r---- memory_layout
0000566634ba7000 4K r---- memory_layout
0000566634ba8000 4K rw--- memory_layout
000056666627e000 132K rw--- [ anon ]
00007618c3000000 160K r---- libc.so.6
00007618c3028000 1632K r-x-- libc.so.6
00007618c31c0000 312K r---- libc.so.6
00007618c320e000 16K r---- libc.so.6
00007618c3212000 8K rw--- libc.so.6
00007618c3214000 52K rw--- [ anon ]
00007618c3415000 12K rw--- [ anon ]
00007618c341e000 8K rw--- [ anon ]
00007618c3420000 16K r---- [ anon ]
00007618c3424000 8K r---- [ anon ]
00007618c3426000 8K r-x-- [ anon ]
00007618c3428000 4K r---- ld-linux-x86-64.so.2
00007618c3429000 188K r-x-- ld-linux-x86-64.so.2
00007618c3458000 44K r---- ld-linux-x86-64.so.2
00007618c3463000 8K r---- ld-linux-x86-64.so.2
```

Observation: The memory organization of the running process was studied using `/proc/<PID>/maps` and the `pmap` command. The virtual memory mappings showed separate regions for the executable code, global/static data, heap, stack, and shared libraries. The heap contained dynamically allocated memory using `malloc()`, while the stack contained local variables. The permissions of each region were also observed using read, write, and execute flags. The virtual addresses may vary between executions due to Address Space Layout Randomization (ASLR).